

Standalone TikZ to PNG with tectonic and pixi

Rohit Goswami · 2026-02-24 · tex · pixi · build

For blog posts and slides I normally embed Graphviz DOT directly in `orgmode` and let the export handle rendering. That works well until a paper schematic needs inline math or a font family that does not cover enough Unicode glyphs [^fn:1]. At that point, TikZ in a `standalone` document class is the path of least resistance: full LaTeX math rendering, arbitrary fonts through the TeX ecosystem, and tightly cropped output without manual trimming.

The build script loops over every `.tex` in a directory, compiles to PDF via `tectonic`, then rasterizes with `pdftoppm` (poppler):

```
#!/usr/bin/env bash
set -euo pipefail

TIKZDIR="v1/pccp/imgs/tikz"
OUTDIR="v1/pccp/imgs/tikz"
DPI="${DPI:-300}"

for texfile in "${TIKZDIR}/*.tex; do
  name="$(basename "${texfile}" .tex)"
  echo "Building ${name}..."
  tectonic "${texfile}" --outdir "${OUTDIR}"
  pdftoppm -png -r "${DPI}" -singlefile \
    "${OUTDIR}/${name}.pdf" "${OUTDIR}/${name}"
  echo "  -> ${OUTDIR}/${name}.png"
done
```

Wiring this into `pixi` keeps the dependencies pinned and the invocation trivial:

```
[tasks]
tikz-figs = { cmd = "bash v1/pccp/imgs/tikz/build_tikz.sh" }

[feature.texbuild.dependencies]
tectonic = "*"
poppler = "*"

[environments]
texbuild = { features = ["texbuild"] }
```

Then `pixi run -e texbuild tikz-figs` rebuilds everything. Override the resolution with `DPI=600 pixi run -e texbuild tikz-figs` for print quality. The same figures can live as `#+begin_export latex` blocks in the `orgmode` paper source for native export, with the standalone PNGs serving as quick-reference copies or targets for non-TeX workflows.